

Parallel For Loop in C# with Examples

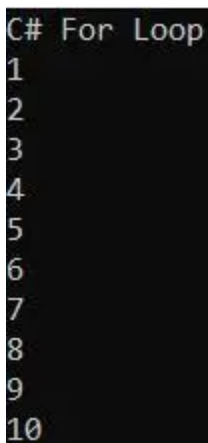
In this article, I will discuss the static **Parallel For Loop in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the basic concepts of [Parallel Programming in C#](#).

Parallel For Loop in C#:

We know that we can use a standard for loop in C# to iterate a block of code, and this execution of the code will be done sequentially. For example, let's say we want to print the numbers from 1 to 10 sequentially, then we can use a for loop as shown in the below example.

```
using System;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("C# For Loop");
            for (int i = 1; i <= 10; i++)
            {
                Console.WriteLine(i);
            }
            Console.ReadLine();
        }
    }
}
```

Output:



```
C# For Loop
1
2
3
4
5
6
7
8
9
10
```

But how did we know that the numbers would be printed in order from one to ten? Well, because our for loop executes sequentially. From the first one, which is one, until the last one, which in this case will be the number ten. But what happens if we want to execute the different iterations of this block of code simultaneously? For that, we can use Parallel For Loop. With Parallel For Loop in C#, we have something very similar to the standard for loop, but the main difference is that with Parallel For Loop, the different executions will be done in parallel. The following example is the parallel version of the previous example.

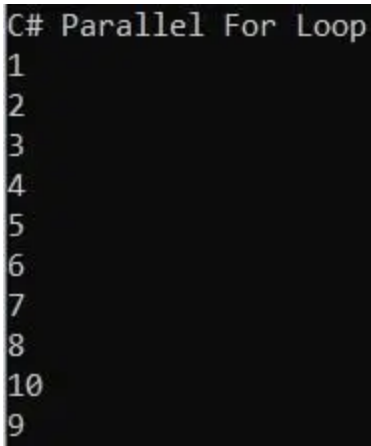
```

using System;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("C# Parallel For Loop");

            //It will start from 1 until 10
            //Here 1 is the start index which is Inclusive
            //Here 11 is the end index which is Exclusive
            //Here number is similar to i of our standard for loop
            //The value will be store in the variable number
            Parallel.For(1, 11, number => {
                Console.WriteLine(number);
            });
            Console.ReadLine();
        }
    }
}

```

With Standard for loop, we can predict the order in which the numbers would appear on the console, but we cannot do that with Parallel For loop. Now, run the application and see the output. Try to run the code multiple times, and you might get different orders of the numbers in the console.



```

C# Parallel For Loop
1
2
3
4
5
6
7
8
10
9

```

What is Parallel For Loop in C#?

In C#, the Parallel For loop is a part of the Task Parallel Library (TPL) and is used for parallelizing loops. It executes iterations of a loop in parallel on multiple threads, potentially improving performance by utilizing multiple processors or cores more effectively. This approach is especially beneficial when executing CPU-bound operations.

What is the Difference Between the Parallel For loop and Standard C# for loop?

The main difference between the Parallel For loop and the standard C# for loop is as follows.

1. In the case of the standard C# for loop, the loop will run using a single thread, whereas, in the case of the Parallel For loop, the loop will execute using multiple threads.
2. The second difference is that, in the case of the standard C# for loop, the loop is iterated in sequential order, whereas, in the case of the Parallel For loop, the order of the iteration is not going to be in sequential order.

Note: When the iterations are independent of each other, it means that subsequent iterations do not need the state updates made by previous iterations. In such cases, we must use Task Parallel Library (TPL) to run each iteration in parallel on all the available cores. Moreover, the iteration should be an expensive iteration; otherwise, we will get a negative performance, which we will also discuss as part of this article.

Syntax:

A sequential for loop in C#:	A parallel for loop in C#:
<pre>int n = 10; for (int i = 0; i < n; i++) { // ... };</pre>	<pre>int n = 10; Parallel.For(0, n, i => { // ... });</pre>

Example to understand the differences between Standard For Loop and Parallel For Loop in C#:

```
using System;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("C# For Loop");
            int number = 10;
            for (int count = 0; count < number; count++)
            {
                //Thread.CurrentThread.ManagedThreadId returns an integer that
                //represents a unique identifier for the current managed thread
                Console.WriteLine($"value of count = {count}, thread = {Thread.CurrentThread.ManagedThreadId}");
                //Sleep the loop for 10 milliseconds
                Thread.Sleep(10);
            }
            Console.WriteLine();
            Console.WriteLine("Parallel For Loop");
            Parallel.For(0, number, count =>
```

```

        {
            Console.WriteLine($"value of count = {count}, thread =
{Thread.CurrentThread.ManagedThreadId}");
            //Sleep the loop for 10 milliseconds
            Thread.Sleep(10);
        });
        Console.ReadLine();
    }
}

```

Once you run the above code, you will get the following output.

```

C# For Loop
value of count = 0, thread = 1
value of count = 1, thread = 1
value of count = 2, thread = 1
value of count = 3, thread = 1
value of count = 4, thread = 1
value of count = 5, thread = 1
value of count = 6, thread = 1
value of count = 7, thread = 1
value of count = 8, thread = 1
value of count = 9, thread = 1

Parallel For Loop
value of count = 0, thread = 1
value of count = 1, thread = 1
value of count = 2, thread = 3
value of count = 4, thread = 5
value of count = 6, thread = 4
value of count = 8, thread = 6
value of count = 3, thread = 1
value of count = 7, thread = 3
value of count = 5, thread = 5
value of count = 9, thread = 4

```

As you can see in the above output, the standard C# for loop iterates sequentially using a single thread. As a result, the results are printed sequentially. On the other hand, you can see with the Parallel for loop, the results are not printed in sequential order. This is because it uses multiple threads to iterate over the collection. You can see that in our example, it uses five threads to execute the code. It may vary in your system.

This means that although we are guaranteed that the 10 executions will happen, we don't know ahead of time the order of the executions of the iterations of the Parallel For Loop, which means that if you have a block of code that you want to iterate over several times, if you want to speed things up and the operations can be done in any order, then you can consider using parallel For Loop in C#.

Example for a Better Understanding From a Performance Point of View.

First, we will write the example using C# for loop and see how long it will take to complete the execution. Then, we will write the same example using the Parallel For method and see how long it will take to complete

the execution.

In the below example, we create a sequential loop. The loop iterates ten times, and the loop control variable increases from zero to nine. In each iteration, the `DoSomeIndependentTask` method is called. The `DoSomeIndependentTask` method performs a calculation that is included to generate a long enough pause to see the performance improvement of the parallel version.

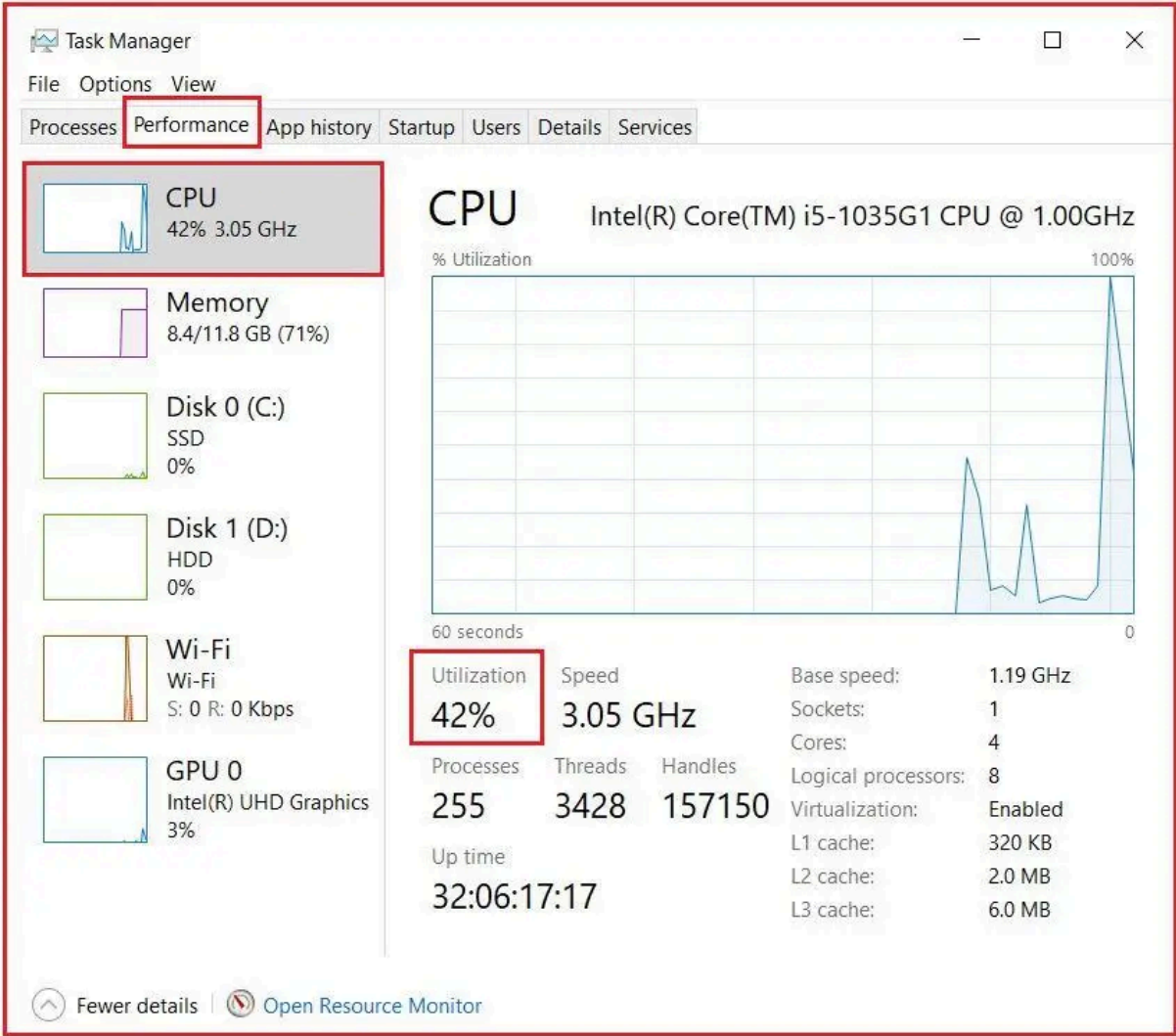
```
using System;
using System.Diagnostics;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main()
        {
            DateTime StartDateTime = DateTime.Now;
            Stopwatch stopWatch = new Stopwatch();
            Console.WriteLine("For Loop Execution start");
            stopWatch.Start();
            for (int i = 0; i < 10; i++)
            {
                long total = DoSomeIndependentTask();
                Console.WriteLine("{0} - {1}", i, total);
            }
            DateTime EndDateTime = DateTime.Now;
            Console.WriteLine("For Loop Execution end ");
            stopWatch.Stop();
            Console.WriteLine($"Time Taken to Execute the For Loop in milliseconds {stopWatch.ElapsedMilliseconds}");

            Console.ReadLine();
        }
        static long DoSomeIndependentTask()
        {
            //Do Some Time Consuming Task here
            //Most Probably some calculation or DB related activity
            long total = 0;
            for (int i = 1; i < 100000000; i++)
            {
                total += i;
            }
            return total;
        }
    }
}
```

Output:

```
For Loop Execution start
0 - 4999999950000000
1 - 4999999950000000
2 - 4999999950000000
3 - 4999999950000000
4 - 4999999950000000
5 - 4999999950000000
6 - 4999999950000000
7 - 4999999950000000
8 - 4999999950000000
9 - 4999999950000000
For Loop Execution end
Time Taken to Execute the For Loop in milliseconds 2317
```

As you can see from the above output, the for-loop statement took approximately 2317 milliseconds to complete the execution. In your machine, the time may vary. Now, do one thing. While running the code, open Task Manager and observe the CPU Utilization. In my machine, a maximum of 42% CPU Utilization is done, as shown in the below image. You need to run the code, and at the same time, you need to observe the CPU Utilization and see what the maximum CPU Utilization is done on your machine.



In the above example, the DoSomeIndependentTask method does not call any external services or APIs, so it is a CPU-bound operation. The CPU-bound operations are those operations whose resolution depends on the processor, not services external to the application. In general, we need to use parallelism in CPU-bound operations. Let's rewrite the same example using the Parallel For method and see the performance.

```
using System;
using System.Diagnostics;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main()
        {
            DateTime StartDateTime = DateTime.Now;
            Stopwatch stopWatch = new Stopwatch();
            Console.WriteLine("Parallel For Loop Execution start");
            stopWatch.Start();

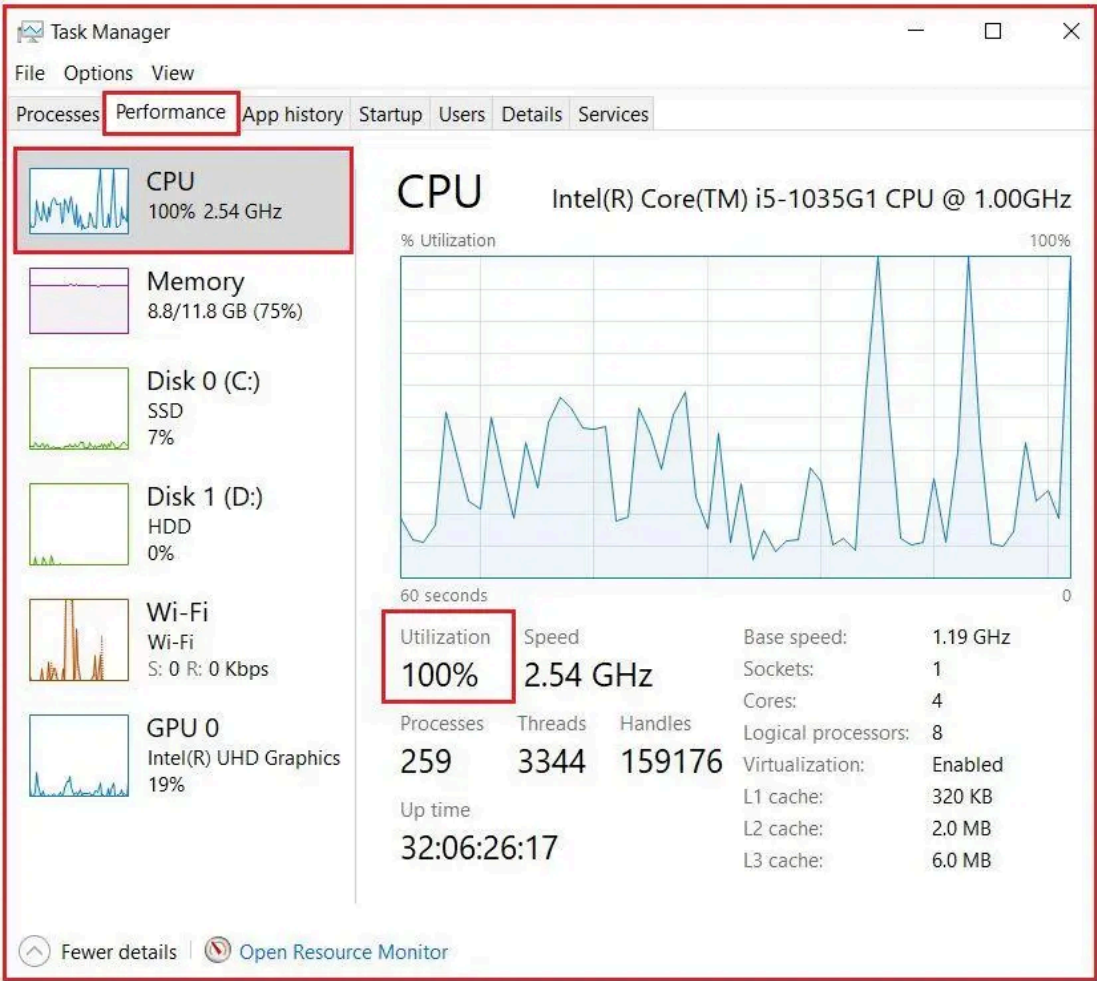
            Parallel.For(0, 10, i => {
                long total = DoSomeIndependentTask();
                Console.WriteLine("{0} - {1}", i, total);
            });
            DateTime EndDateTime = DateTime.Now;
            Console.WriteLine("Parallel For Loop Execution end ");
            stopWatch.Stop();
            Console.WriteLine($"Time Taken to Execute Parallel For Loop in milliseconds: {stopWatch.ElapsedMilliseconds}");

            Console.ReadLine();
        }
        static long DoSomeIndependentTask()
        {
            //Do Some Time Consuming Task here
            //Most Probably some calculation or DB related activity
            long total = 0;
            for (int i = 1; i < 100000000; i++)
            {
                total += i;
            }
            return total;
        }
    }
}
```

Output:

```
Parallel For Loop Execution start
7 - 4999999950000000
5 - 4999999950000000
2 - 4999999950000000
6 - 4999999950000000
0 - 4999999950000000
3 - 4999999950000000
4 - 4999999950000000
1 - 4999999950000000
8 - 4999999950000000
9 - 4999999950000000
Parallel For Loop Execution end
Time Taken to Execute Parallel For Loop in milliseconds 793
```

As shown in the above output, the Parallel For method took 793 milliseconds to complete the execution compared to the 2317 milliseconds with the standard for loop execution. Again, do the same thing. While running the code, open Task Manager and observe the CPU Utilization. In my machine, a maximum of 100% CPU Utilization is done at some point while running the code, as shown in the image below. You need to run the code, and at the same time, you need to observe the CPU Utilization and see what the maximum CPU Utilization is done on your machine.



So, the parallel version of the For loop performs better than the standard for loop. But this is not always true. Sometimes, the standard for loop will give you better performance than the Parallel For loop, which we will

discuss in our upcoming article.

ParallelOptions Class in C#

The ParallelOptions class is one of the most useful classes for multithreading. This class provides options to limit the number of concurrently executing loop methods.

The Degree of Parallelism in C#:

Using the Degree of Parallelism, we can specify the maximum number of threads to be used to execute the program. Following is the syntax to use the ParallelOptions class with a Degree of Parallelism.

```
var options = new ParallelOptions()
{
    MaxDegreeOfParallelism = 2
};

int n = 10;
Parallel.For(0, n, options, i =>
{
    //...
});
```

The MaxDegreeOfParallelism property affects the number of concurrent operations run by Parallel method calls that are passed this ParallelOptions instance. A positive property value limits the number of concurrent operations to the set value. If it is -1, there is no limit on the number of concurrently running operations.

Example to Understand MaxDegreeOfParallelism in C#

```
using System;
using System.Threading;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Limiting the maximum degree of parallelism to 2
            var options = new ParallelOptions()
            {
                MaxDegreeOfParallelism = 2
            };
            int n = 10;
            Parallel.For(0, n, options, i =>
            {
                Console.WriteLine(@"value of i = {0}, thread = {1}",
```

```

        i, Thread.CurrentThread.ManagedThreadId);
        Thread.Sleep(10);
    });
    Console.WriteLine("Press any key to exist");
    Console.ReadLine();
}
}
}

```

Output:

```

value of i = 0, thread = 9
value of i = 5, thread = 10
value of i = 6, thread = 10
value of i = 1, thread = 9
value of i = 7, thread = 10
value of i = 2, thread = 9
value of i = 8, thread = 10
value of i = 3, thread = 9
value of i = 9, thread = 10
value of i = 4, thread = 9
Press any key to exist

```

As we set the degree of parallelism to 2. So, a maximum of 2 threads is used to execute the code, as seen from the above output.

Terminating a Parallel For Loop in C#:

The example below shows how to break out of a For loop and stop a loop. In this context, “break” means completing all iterations on all threads before the current iteration on the current thread and then exiting the loop. “Stop” means to stop all iterations as soon as convenient.

```

using System;
using System.Linq;
using System.Threading.Tasks;
namespace ParallelProgrammingDemo
{
    class Program
    {
        static void Main()
        {
            var BreakSource = Enumerable.Range(0, 1000).ToList();
            int BreakData = 0;
            Console.WriteLine("Using loopstate Break Method");
            Parallel.For(0, BreakSource.Count, (i, BreakLoopState) =>
            {
                BreakData += i;
                if (BreakData > 100)

```

```

        {
            BreakLoopState.Break();
            Console.WriteLine("Break called iteration {0}. data = {1} '
BreakData);
        }
    });
    Console.WriteLine("Break called data = {0} ", BreakData);
    var StopSource = Enumerable.Range(0, 1000).ToList();
    int StopData = 0;
    Console.WriteLine("Using loopstate Stop Method");
    Parallel.For(0, StopSource.Count, (i, StopLoopState) =>
    {
        StopData += i;
        if (StopData > 100)
        {
            StopLoopState.Stop();
            Console.WriteLine("Stop called iteration {0}. data = {1} '
StopData);
        }
    });
    Console.WriteLine("Stop called data = {0} ", StopData);
    Console.ReadKey();
}
}
}

```

Output:

```

Using loopstate Break Method
Break called iteration 14. data = 105
Break called data = 105
Using loopstate Stop Method
Stop called iteration 14. data = 105
Stop called iteration 125. data = 230
Stop called data = 230

```

In Parallel For or Parallel ForEach Loop in C#, you cannot use the same break or Exit statement used in a sequential loop because those language constructs are valid for loops, and a parallel “loop” is actually a method, not a loop. Instead, you use either the Stop or Break method.

Considerations and Best Practices

- **Thread Safety:** Ensure that your loop code is thread-safe. Avoid modifying shared data without proper synchronization mechanisms like locks, concurrent collections, or other thread synchronization techniques.
- **Order of Execution:** The iterations of a Parallel For loop do not run in a guaranteed order, and different iterations may run simultaneously. This is different from a regular for loop where each iteration runs sequentially.

- **Exception Handling:** Handling exceptions in a Parallel For loop can be more complex. You might use a `ParallelLoopState` object to stop or break the loop, and an `AggregateException` may be thrown if multiple iterations throw exceptions.
- **Performance:** While `Parallel.For` can improve performance for CPU-bound operations, it might not be beneficial for I/O-bound operations. Additionally, for very short or simple loop bodies, the overhead of parallelization may outweigh its benefits.
- **Cancellation Support:** `Parallel.For` support task cancellation using the `CancellationToken` class. This can be useful for long-running operations that need to be stopped under certain conditions.
- **Return Values and Loop Control:** `Parallel.For` returns, a `ParallelLoopResult` which can be used to determine if the loop ran to completion or was stopped/prematurely exited. You can also control the loop execution using the `ParallelLoopState` object in the loop body.

Alternatives

- **PLINQ:** For operations on collections, PLINQ (Parallel LINQ) can be a more straightforward alternative, allowing LINQ queries to run in parallel.
- **Asynchronous Programming:** For I/O-bound operations, consider using asynchronous programming (`async` and `await`) instead of parallel loops.

In the next article, I will discuss the [Parallel ForEach Method in C#](#) with Examples. In this article, I try to explain the `Parallel For` in C# with some examples. I hope you understand the need and use of the `Parallel For` method in C#.